README.md

My First Repository

Description

This is my first GitHub repository. I created it to learn how to use GitHub to store files and track changes.

Objectives

- Learn how to create a GitHub repository.
- Learn how to edit a README.md file.
- Learn how to make a commit.

Steps Completed

1. Created a public GitHub repository.
2. Added a README.md file.
3. Edited the README content.
4. Saved the changes using a commit.

Conclusion

This exercise helped me understand how to create a GitHub repository, edit files, and save changes using commits.

Exercise B: API Reference Entry

Endpoint Name: software engineering

Purpose: This API allows an authenticated user to create a new task in a project management application.

POST

`/api/v1/projects/{projectId}/tasks`

This endpoint creates a new task inside a specified project.

Parameter	Data type	Required?	Description
`projectId`	String	Yes	Identifies the project.
`title`	String	Yes	Name of the task.
`description`	String	No	Additional task details.
`assigneeId`	String	Yes	ID of the user assigned the task.

`dueDate`	Date	Yes	Deadline for completing the task.
`priority`	String	Yes	Task importance: low, medium, or high.

`Authorization: Bearer <access_token>` — verifies the user's identity.
`Content-Type: application/json` — tells the server that the request body is JSON.
`Accept: application/json` — requests a JSON response.

```
{
  "title": "Prepare monthly project report",
  "description": "Collect progress updates and prepare the report.",
  "assigneeId": "user_204",
  "dueDate": "2026-11-15",
  "priority": "high"
}
```

```
{
  "id": "task_789",
  "projectId": "proj_101",
  "title": "Prepare monthly project report",
  "description": "Collect progress updates and prepare the report.",
  "assigneeId": "user_204",
  "dueDate": "2026-11-15",
  "priority": "high",
  "status": "open",
  "createdAt": "2026-10-11T10:30:00Z",
  "updatedAt": "2026-10-11T10:30:00Z"
}
```

Code	Meaning

`201 Created`	Task created successfully.
`400 Bad Request`	Request contains invalid data.
`401 Unauthorized`	Authentication token is missing or invalid.
`403 Forbidden`	User lacks permission to create the task.
`404 Not Found`	Project or assignee cannot be found.
`409 Conflict`	Request conflicts with a project rule.
`415 Unsupported Media Type`	Request does not use the supported JSON format.
`422 Unprocessable Content`	Data fails a business validation rule.

`429 Too Many Requests`	Too many requests were sent.
`500 Internal Server Error`	An unexpected server error occurred.

Validation Rules

1. The title, assignee ID, due date, and priority are required.
2. The title must not be empty.
3. The due date must use the YYYY-MM-DD format.
4. The priority must be low, medium, or high.
5. The project and assignee must exist.
6. The authenticated user must have permission to create tasks in the project.

Conclusion

The Create Task API enables authenticated users to create tasks in a project management application. This reference explains the endpoint, request parameters, authentication headers, JSON examples, response codes, and validation rules.

Exercise C

Executive Summary: Database Evaluation for a University Student Records Management System

The purpose of this four-week evaluation was to compare MySQL, PostgreSQL, and MongoDB for a new university student records management system. The evaluation focused on storing structured student information, including personal details, enrolment records, and grades, supporting up to 200 staff users simultaneously, ensuring data integrity, and providing manageable maintenance for a small IT team.

Based on the evaluation criteria, **PostgreSQL is recommended** for the university's student records management system. It provides reliable transaction processing and strong support for structured data, making it suitable for maintaining accurate student records and grades.

Two key factors influenced this recommendation. First, **data integrity and transaction reliability** are essential to ensure that student information remains accurate and consistent when multiple staff members access or update records. Second, **ease of maintenance** is important because the university has a small IT team with limited database expertise. PostgreSQL offers a good balance of reliability, structured data management, and maintainability.

The university should adopt PostgreSQL, subject to a practical pilot test to confirm that it meets performance requirements and to provide staff with appropriate training before full implementation.

Exercise D: Structured Peer Critique

Name of exercise: Peer Review of Three Writing Samples

Instructions: The following is a model peer critique. Because your classmate's actual documents have not been provided, check each comment against their work before submitting it.

1. User Manual Procedure

Dimension 1: One action per step

The procedure is easy to follow when each numbered step describes one clear action. If a step combines actions, such as entering a repository name and selecting its visibility, separate them into individual steps to make the instructions easier for beginners.

Dimension 2: Prerequisites

The prerequisites should identify the computer, internet connection, web browser, and GitHub account needed before beginning. Listing these requirements clearly helps a first-semester student prepare without stopping halfway through the procedure.

Dimension 3: Expected results

Each step should explain what the user should see after completing the action. For example, after creating a repository, the expected result should state that the new repository page opens and displays its files.

Dimension 4: Troubleshooting

The troubleshooting section should address a common beginner problem, such as being unable to find the Edit button or receiving a permission error. It should explain the likely cause and provide practical steps for resolving the problem.

2. API Reference Entry

Dimension 1: Completeness of endpoint documentation

The API reference should include the HTTP method, endpoint path, request parameters, required headers, authentication requirements, and all applicable response codes. Each required field should have a data type, a required or optional designation, and a clear description of its purpose.

Dimension 2: JSON formatting and consistency

The example request and response should use valid JSON with correctly quoted property names and string values. The response should consistently reflect the submitted title, description, assignee, due date, and priority while including generated information such as the task ID and creation timestamp.

Dimension 3: Ability to make a successful API call

A developer should be able to construct a valid request using only the endpoint documentation and its examples. The reference should provide the authentication format, complete request body, content type, and successful response so that the developer does not need to guess any required information.

3. Technical Report Executive Summary

Dimension 1: Self-contained summary

The executive summary should explain the purpose of the evaluation, the three databases compared, and the university's student records requirements. It should define unfamiliar technical terms in simple language so administrators can understand the findings without reading the full report.

Dimension 2: Recommendation and justification

The summary should clearly identify one recommended database and explain why it best meets the university's needs. The justification should connect the recommendation to data integrity, support for 200 concurrent staff users, and the small IT team's maintenance capabilities.

Dimension 3: Appropriate language for a non-technical audience

The summary should use professional, straightforward language that university administrators can understand. Technical terms such as ACID compliance should be explained in terms of keeping student records accurate and consistent rather than being presented without explanation.

Dimension 4: Word limit

The executive summary must not exceed 200 words while still stating the purpose, scope, recommendation, and two most important decision factors. The final draft should be checked for its word count and edited to remove repetition or unnecessary technical detail.

Final submission checklist

- Exercise C: Include the executive summary in your document.
- Exercise D: Review your classmate's actual three writing samples and use specific feedback that accurately describes their work.
- Ensure that each of the four dimensions in each critique section contains two sentences.
- Submit Exercises C and D according to your instructor's submission instructions.